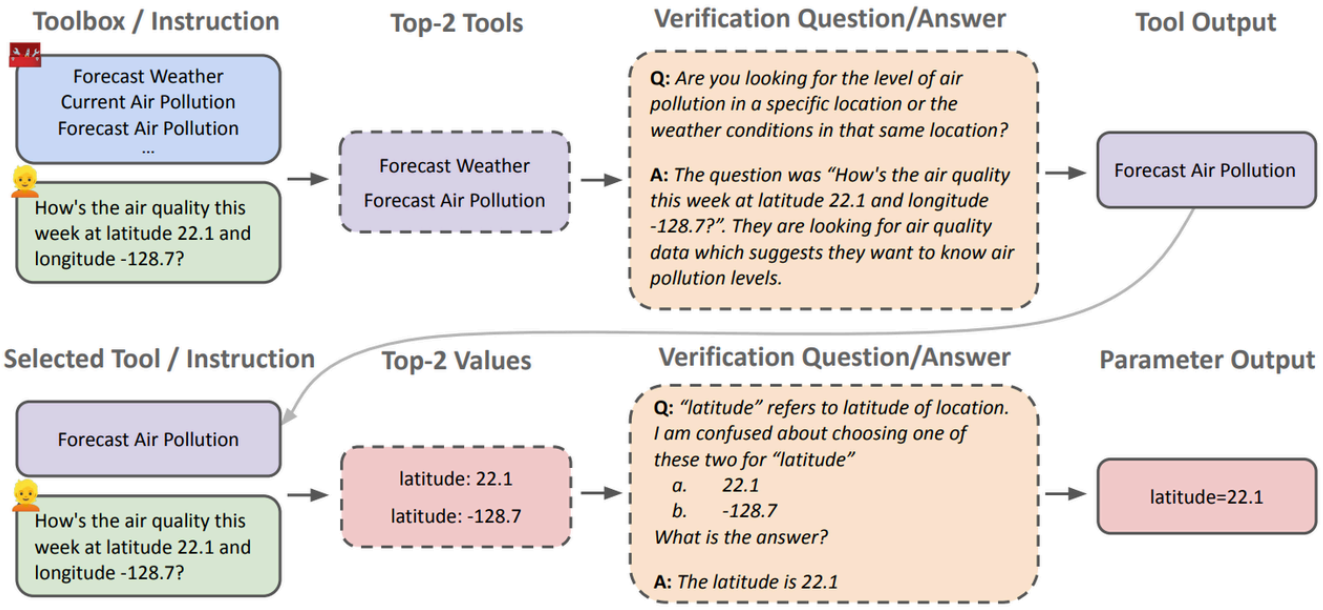
TOOLVERIFIER: Enhancing Tool Use via Contrastive Self-Verification

> #Tools #Inference #Dataset #Finetune



- What problem does this framework solve?
  - LLMs are not able to use tools effectively and robustly with the limited number of demos.
  - This framework provides a complete pipeline of training data synthesis, finetuning, verification, and inference.
- What does this framework do?
  - In a nutshell, this framework has 2 parts: Tool-wise and Parameter-wise pipelines
  - Tools
    - This part includes a dataset generation for tool-selection training and tool-selection verification.
    - **Goal**: to teach a model to select tools with only tool names and introductions instead of tons of demonstrations.
    - Initially, manually create some **seeding tools** for the repo (*8 tools on paper*).
    - Let an LLM, with the **seeding tools** as "few-shots", generate more tools.
    - **Ensure diversity**: Use an embedding model (*e.g., RoBERTa*) to calculate the similarity of the newly added tool and existing tools. Eliminate those too similar ones, and add the instruction of generating different types of tools back to the tool-maker model.
    - **More Challenging**: For each generated existing tool, generate some (*2 on paper*) similar tools with *subtle differences* (*e.g., weather forecast tool vs current weather tool*).
    - **Training set generation**:
      - For each tool, generate some instructions (user query) (*3 on paper*).
      - **Normal samples**: For each instruction, select a set of tools with the ground truth in.

        - **Hard samples**: For each instruction, select a set that contains ONLY the ground truth and the similar tools (with no other obvious wrong tools).
        - To enhance the reasoning, for each data point, write a reason to explain **why choose the ground truth tool**.
    - **Tool Verification** (inference time): it is observed that those correct tool selection predictions typically appear in the top few predictions. Let the **finetuned model** select one tool, remove that tool, and let it select again to form a **top-2** selection question. Plus, use the **original user query** as the problem. Let the model make the second selection (*as it is proofed that reflection can eliminate hallucinations*).
  - Parameters (All inference time)
    - **Parameter generation**
    - **Parameter Verification**: In addition to the parameter generated by the **finetuned model**, let another model generate a set of parameters with few shots. Form an MCQ using the generated parameters and the user's original input and let the **finetuned model** have a choice.
- My Thought
  - This framework can be used to optimize this self-evolving framework.
  - Though directly importing everything may violate the self-evolving nature, some philosophies can be used.
  - Split the manger's role into 2: **tool selector** and manager, where the former concentrates on only tool finding and parameter deciding and the latter one is only for agent coordination.
  - The **tool selector** agent can be **pre-finetuned** before being assembled into the framework to increase its ability of selecting tools with a **more broad training set** (such as including a situation where there is no proper tool, in which case the tool selector will tell the manager).
  - The tool-selection verification and parameter verification can also be introduced to optimize the tool-selector agent.
  - If tools are not available, tell the manager
- Work Cited
  - Paper
  - Chat History